Kubernetes Local Lab using VirtualBox

Contents

Kubernetes Local Lab 1

K8S installation using VirtualBox 2

Prerequisite: 2

Require host configuration: 2

✅ Kubernetes Requirements Checklist 10

Prepare to start Kubernetes configuration 18

Start Kubernetes configuration 20

✅ Run on worker nodes: 20

✅ Flannel: 20

✅ Calico configuration: 24

✅ Cilium 25

✅ Calico + Istio installation 28

✅ Cilium + Istio installation 32

✅ How To Access the Application 34

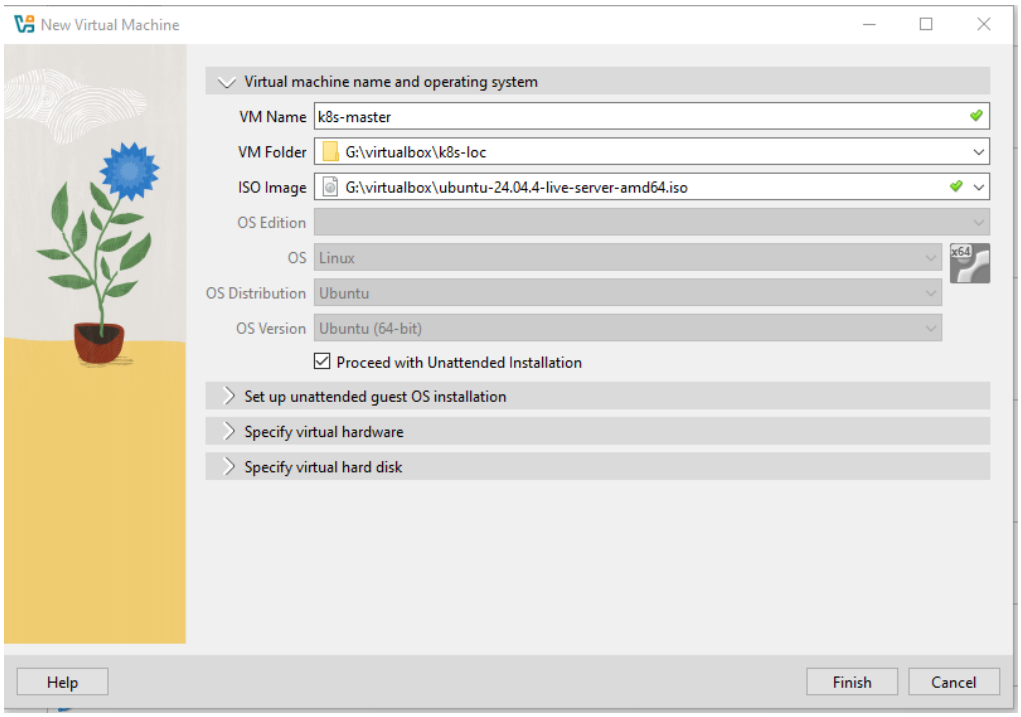
K8S installation using VirtualBox

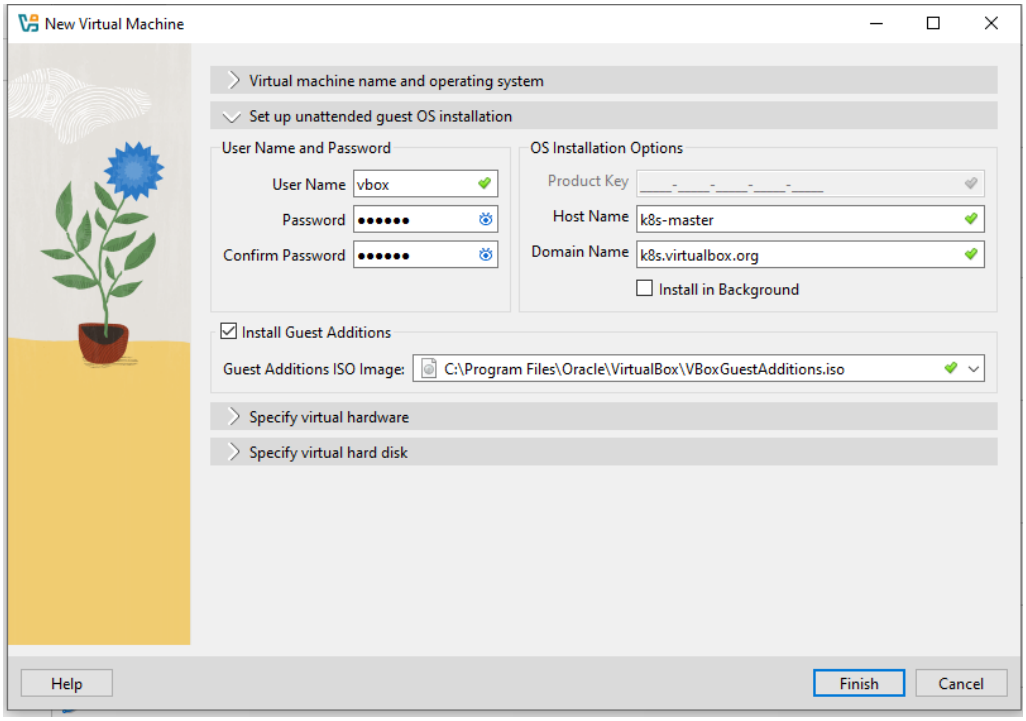
Prerequisite:

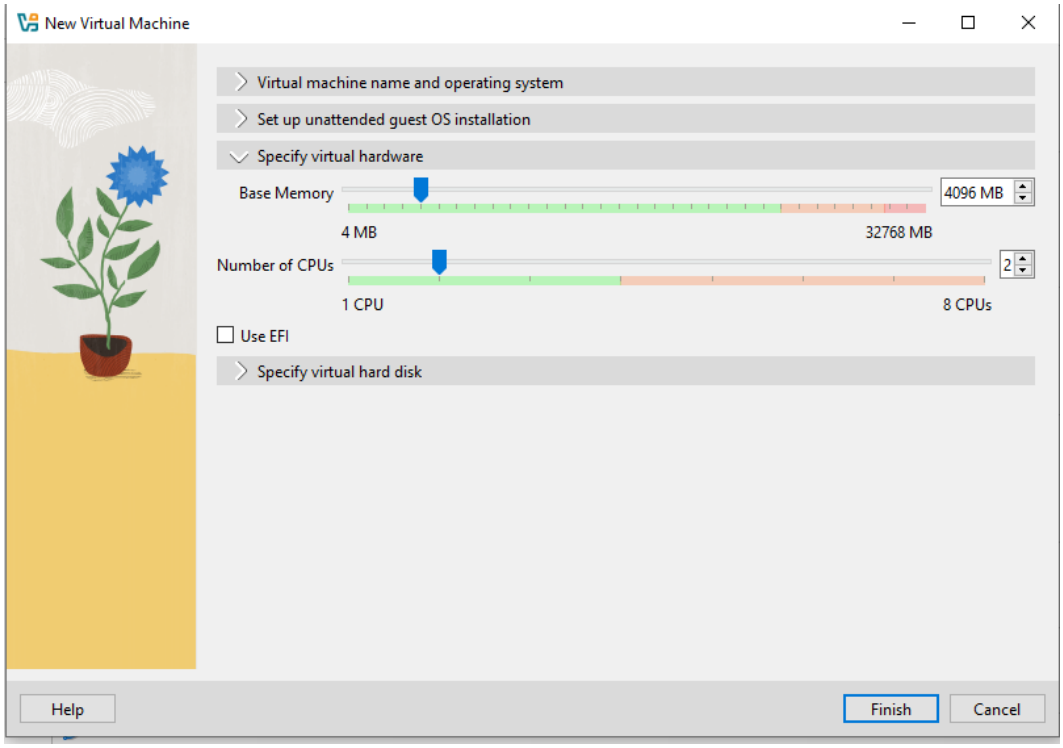
VirtualBox installation on your local comp
Download Ubuntu Server version without GUI -> https://ubuntu.com/download/server

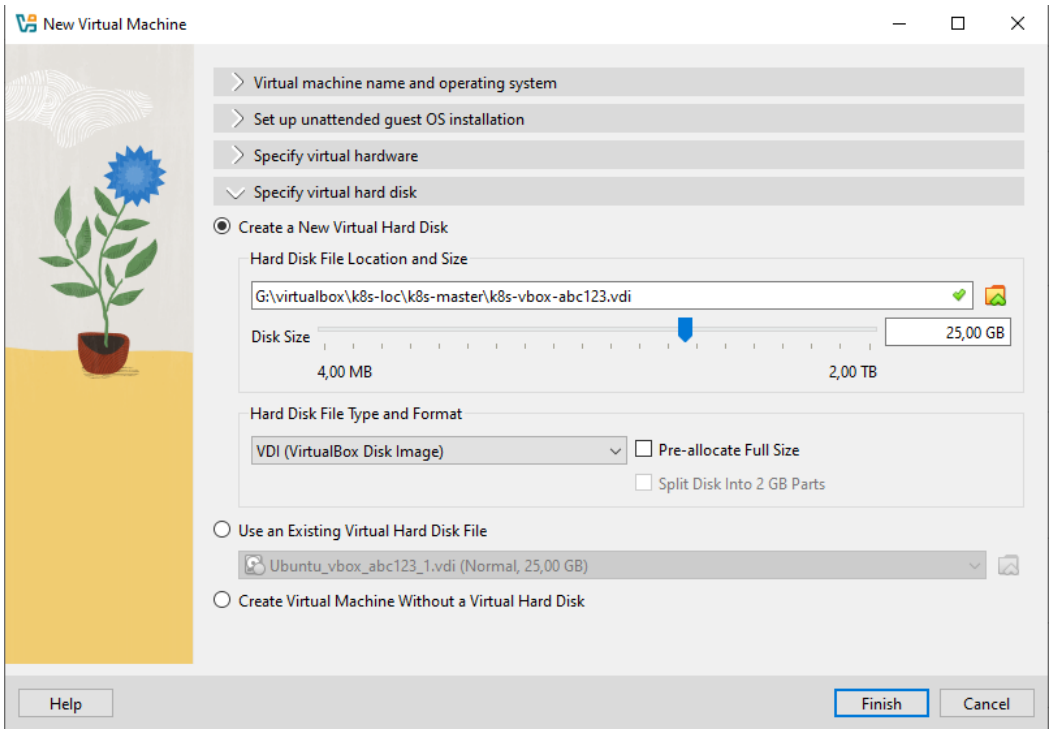
Require host configuration:

CPU: 8
Memory: 32 GB RAM
HDD: 100 GB or more
Create VM inside virtual box:





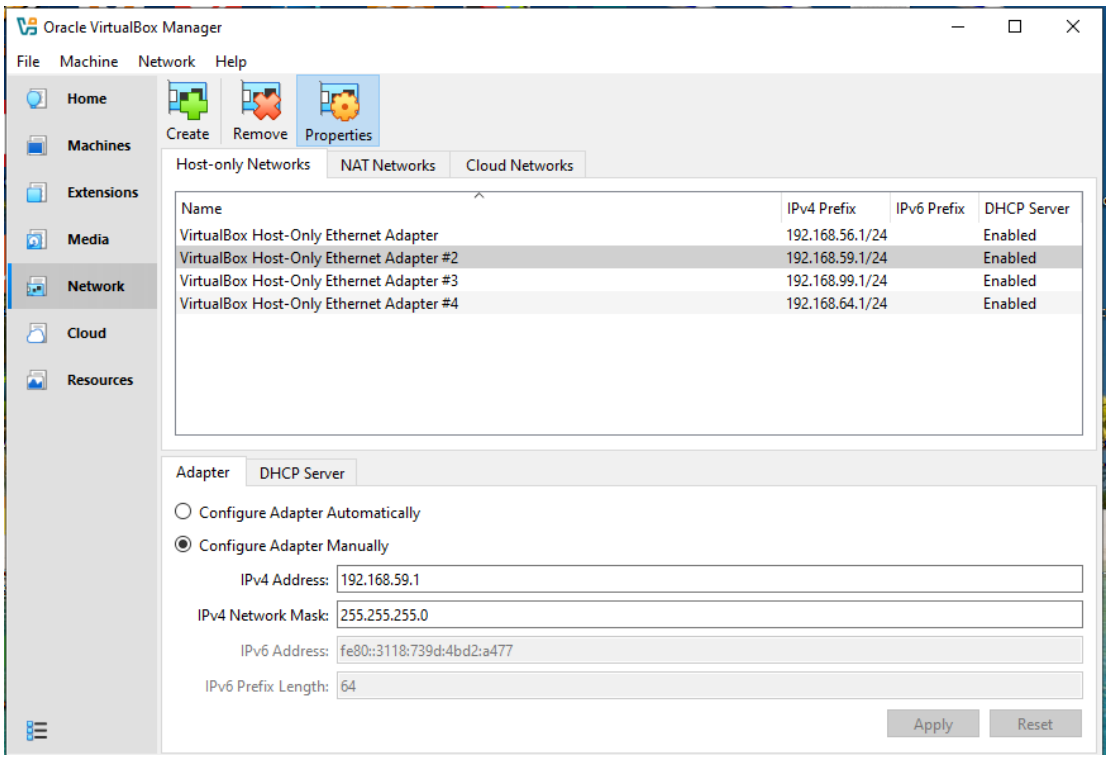




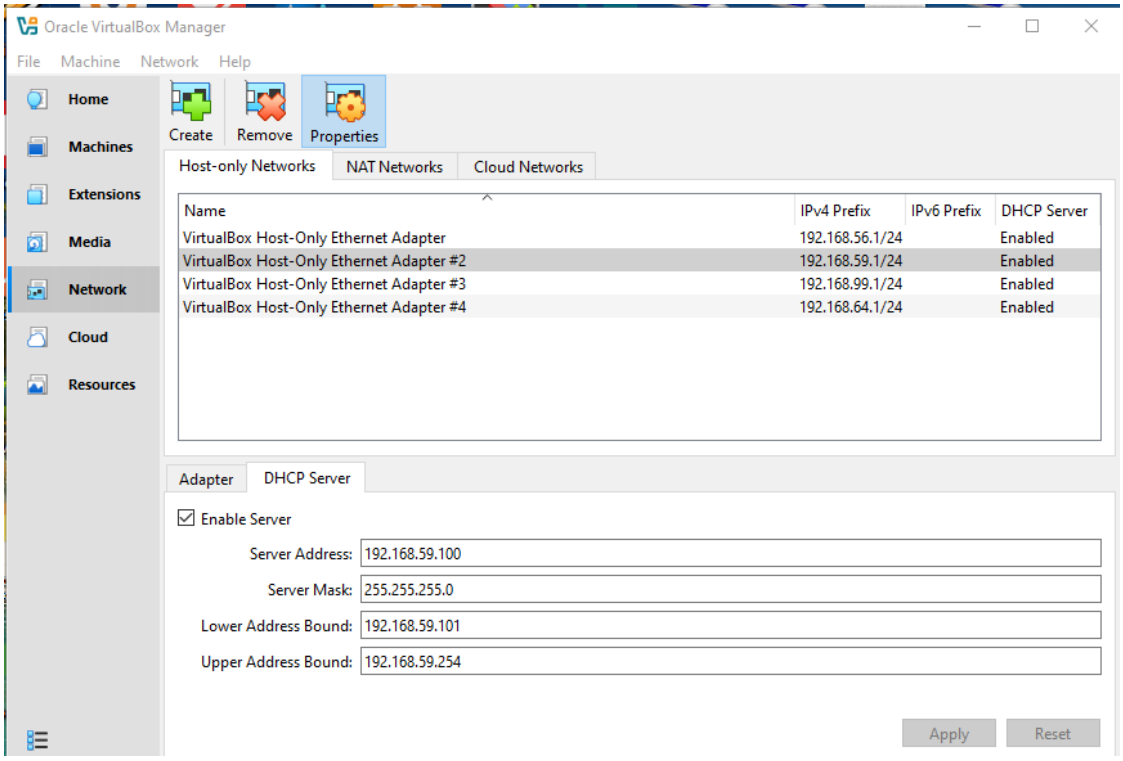
Note:
Installation on my computer take approximately around 10 minutes to finish.
After finish on open terminal, run:
sudo reboot

Log in again using:
user: vbox
password: abc123
Run:
sudo poweroff

We need to add additional NIC to VM of type: Host-only Adapter.
Before to that, inside your virtual box go to Network configuration and check what Host-only Network already exists inside your virtual box.



Check DHCP Server configuration



I will use “VirtualBox Host-Only Ethernet Adapter #2” configuration, with the following parameters:

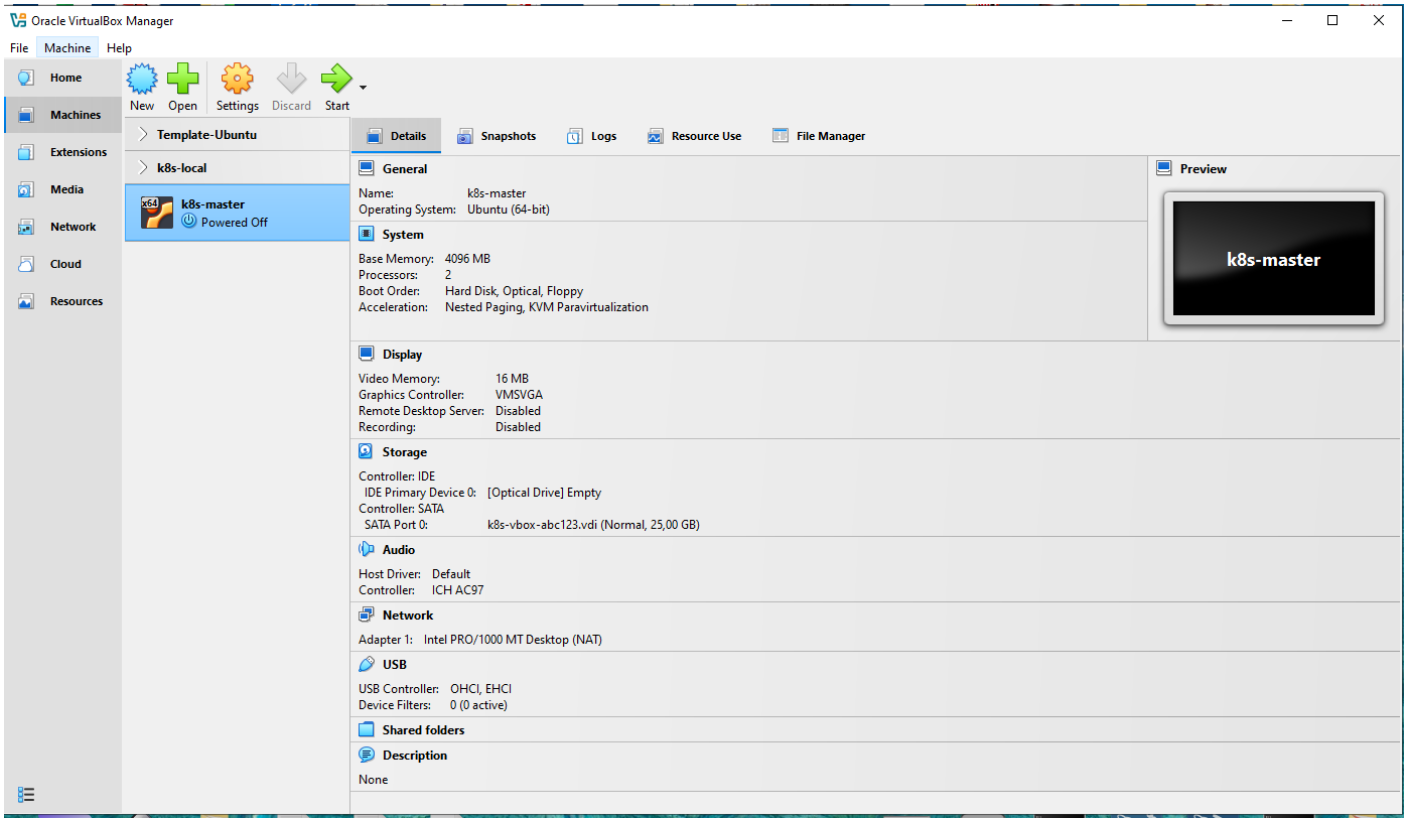
Adapter

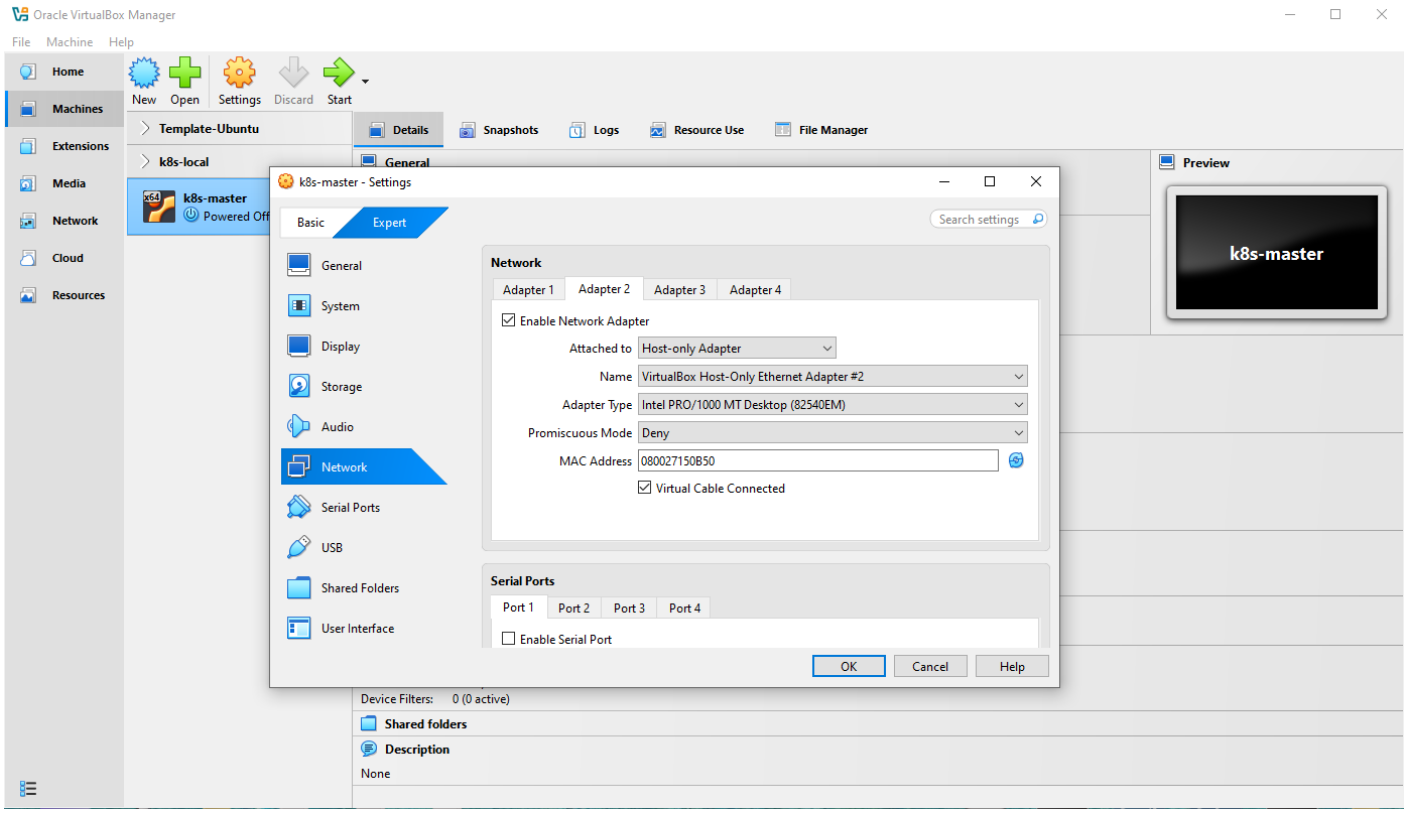
IPv4 Address: 192.168.59.1

IPv4 Network Mask: 255.255.255.0

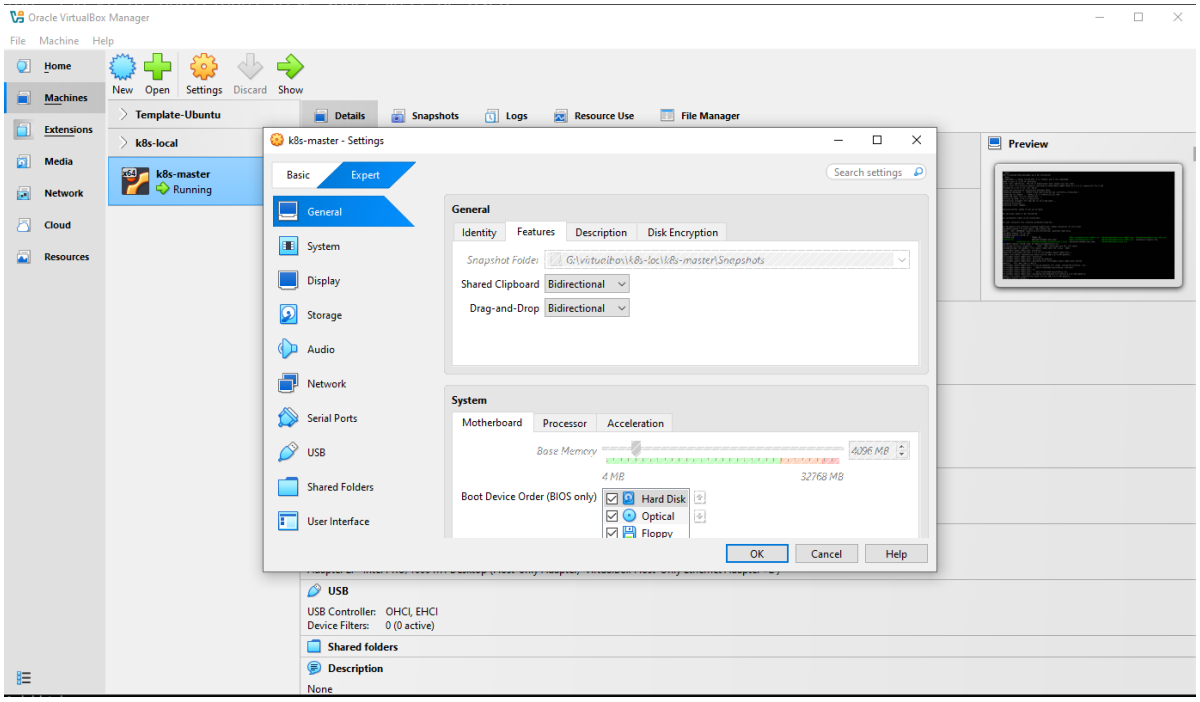
DHCP Server

Server Address: 192.168.59.100
Server Mask: 255.255.255.0
Lower Address Bound: 192.168.59.101
Upper Address Bound: 192.168.59.254





Start VM
Open VM, Settings, General, Features and enable “Shared clipboard”, and “Drag-and-Drop”



If To enable VirtualBox guest tool needs to be installed manually:

✅ Step 1 – Install Required Packages in Ubuntu

Start your Ubuntu VM and update packages:

sudo apt update

sudo apt upgrade -y

Install required build tools:

sudo apt install build-essential dkms linux-headers-$(uname -r) -y

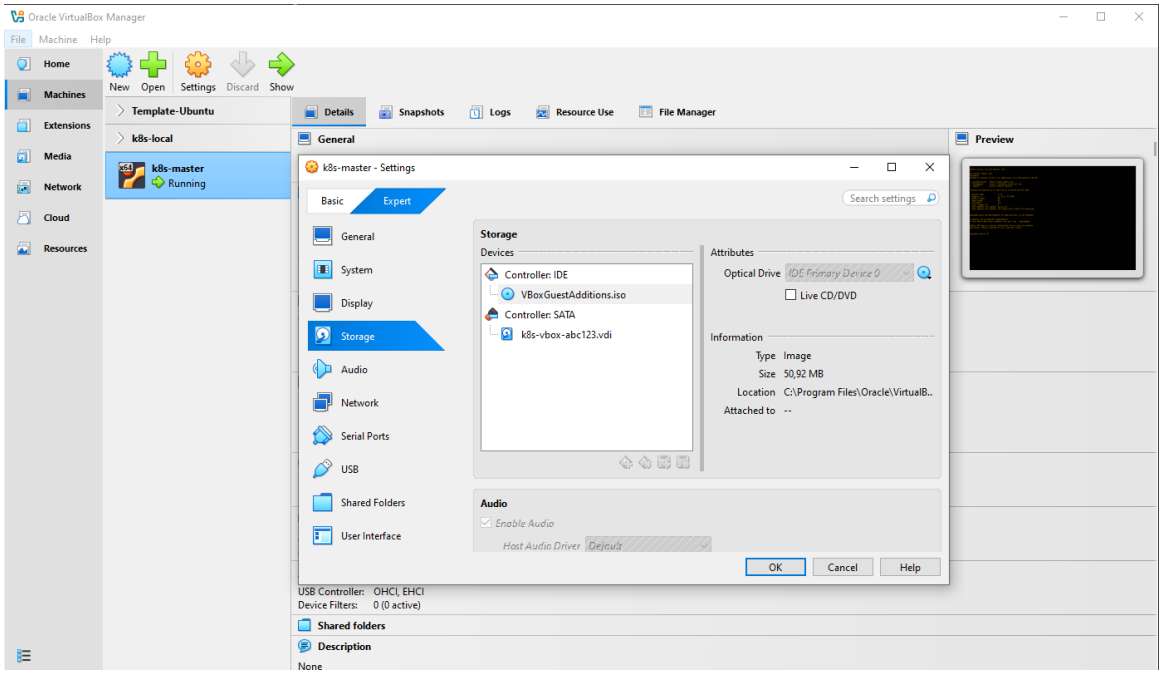
These are required to compile the VirtualBox kernel modules.

✅ Step 2 – Insert Guest Additions CD

In VirtualBox menu:

Devices → Insert Guest Additions CD Image

If prompted to download it, click **Download**.



✅ **Step 3 – Mount the CD (Server / CLI method)**
If you're using Ubuntu Server (no GUI):
sudo mount /dev/cdrom /mnt
Then run:
cd /mnt
sudo sh VBoxLinuxAdditions.run

Wait until installation completes.

✅ **Step 4 – Reboot the VM**
sudo reboot

✅ **Step 5 – Verify It Works**
After reboot:
lsmod | grep vbox
You should see modules like:

- vboxguest
- vboxsf
- vboxvideo

✅ Install OpenSSH Server on Ubuntu
Inside your Ubuntu Server VM:
sudo apt update
sudo apt install openssh-server -y

Check that SSH service is running:
sudo systemctl status ssh

If needed, enable it at boot:
sudo systemctl enable ssh

✅ To disable firewall on ubuntu
On **Ubuntu**, the default firewall is **UFW (Uncomplicated Firewall)**.

1️⃣ **Check Firewall Status**
sudo ufw status
If active, you'll see:
Status: active

2️⃣ **Disable the Firewall**
To stop it immediately:
sudo ufw disable
You should see:
Firewall stopped and disabled on system startup

3️⃣ **Verify It Is Disabled**
sudo ufw status
It should now show:
Status: inactive

⚠️ **Important (Especially for Server)**
If your Ubuntu Server is exposed to:

- Bridged network

- Public IP
- Cloud environment

Disabling firewall completely is **not recommended**.
Instead, you can just allow SSH:
sudo ufw allow ssh

🔎 **If UFW Is Not Installed**
Check:
which ufw
If nothing returns, then firewall is not enabled via UFW.

✅ Set static IP
sudo nano /etc/netplan/01-network-manager-all.yaml
network:
 version: 2
 renderer: networkd
 ethernets:
  enp0s3:
   dhcp4: true
  enp0s8:
   dhcp4: false
   addresses:
    - 192.168.59.10/24
or
    - 192.168.59.11/24
or
    - 192.168.59.12/24

#Apply Changes
sudo netplan generate
sudo netplan apply
ip a

```
vbox@k8s-master:~$ sudo netplan generate
vbox@k8s-master:~$ sudo netplan apply
vbox@k8s-master:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:49:bc:e8 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
       valid_lft 86398sec preferred_lft 86398sec
    inet6 fd17:625c:f037:2:a00:27ff:fe49:bce8/64 scope global dynamic mngtmpaddr noprefixroute
       valid_lft 86399sec preferred_lft 14399sec
    inet6 fe80::a00:27ff:fe49:bce8/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:15:0b:50 brd ff:ff:ff:ff:ff:ff
    inet 192.168.59.10/24 brd 192.168.59.255 scope global enp0s8
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe15:b50/64 scope link
       valid_lft forever preferred_lft forever
vbox@k8s-master:~$
```

On your host open terminal window and try to ping address you set for your Ubuntu VM in previous step

```
Command Prompt
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\vladan>ping 192.168.59.10

Pinging 192.168.59.10 with 32 bytes of data:
Reply from 192.168.59.10: bytes=32 time=2ms TTL=64
Reply from 192.168.59.10: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.59.10:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 2ms, Average = 1ms
Control-C
^C
C:\Users\vladan>
```

To be sure that static IP for Ubuntu VM apply successfully, on host start ping command

```
C:\Users\vladan>ping 192.168.59.10 -t

Pinging 192.168.59.10 with 32 bytes of data:
Reply from 192.168.59.10: bytes=32 time<1ms TTL=64
Reply from 192.168.59.10: bytes=32 time<1ms TTL=64
Reply from 192.168.59.10: bytes=32 time<1ms TTL=64
Reply from 192.168.59.10: bytes=32 time<1ms TTL=64
Reply from 192.168.59.10: bytes=32 time=1ms TTL=64
Reply from 192.168.59.10: bytes=32 time=1ms TTL=64
Reply from 192.168.59.10: bytes=32 time<1ms TTL=64
Reply from 192.168.59.10: bytes=32 time=1ms TTL=64
Reply from 192.168.59.10: bytes=32 time<1ms TTL=64
Reply from 192.168.59.10: bytes=32 time<1ms TTL=64
Reply from 192.168.59.10: bytes=32 time<1ms TTL=64
Reply from 192.168.59.10: bytes=32 time=1ms TTL=64
Request timed out.
Request timed out.
Request timed out.
Request timed out.
Request timed out.
Reply from 192.168.59.10: bytes=32 time=2ms TTL=64
Reply from 192.168.59.10: bytes=32 time<1ms TTL=64
Reply from 192.168.59.10: bytes=32 time=1ms TTL=64
Reply from 192.168.59.10: bytes=32 time=3ms TTL=64
Reply from 192.168.59.10: bytes=32 time<1ms TTL=64
Reply from 192.168.59.10: bytes=32 time<1ms TTL=64
Reply from 192.168.59.10: bytes=32 time<1ms TTL=64
Reply from 192.168.59.10: bytes=32 time<1ms TTL=64
Reply from 192.168.59.10: bytes=32 time<1ms TTL=64
```

and on Ubuntu VM execute:

sudo reboot

If ping from host to Ubuntu VM is working, you can test ssh connection from host to Ubuntu VM using:

ssh vbox@192.168.59.10

```
C:\Users\vladan>ssh vbox@192.168.59.10
The authenticity of host '192.168.59.10 (192.168.59.10)' can't be established.
ED25519 key fingerprint is SHA256:b/pMRgDtQKOXLoBKovLUcySuwReXLEW1cY8166g/wes.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.59.10' (ED25519) to the list of known hosts.
vbox@192.168.59.10's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-100-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat Feb 21 02:02:04 PM UTC 2026

  System load:             0.02
  Usage of /:              11.9% of 24.44GB
  Memory usage:            6%
  Swap usage:              0%
  Processes:               122
  Users logged in:         1
  IPv4 address for enp0s3: 10.0.2.15
  IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe49:bce8


Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


vbox@k8s-master:~$
```

Edit /etc/hosts on each VM
On k8s-master:
#Add entries for all VMs in the internal network:
sudo nano /etc/hosts
127.0.0.1 k8s-master
localhost k8s-master
192.168.59.10 k8s-master
192.168.59.11 k8s-worker1
192.168.59.12 k8s-worker2

✅ Kubernetes Requirements Checklist

Before running kubeadm init on the master:

Requirement	Status	/ Notes
OS update & basic packages	✅	You already included curl, vim, git
Disable swap	✅	Must be off on all nodes, otherwise kubelet refuses to start
Time sync	✅	Using chrony is correct
Network connectivity	✅	All VMs can ping each other (internal network)
Hostname	✅	Unique (k8s-master, k8s-worker1, k8s-worker2)
/etc/hosts entries	✅	Optional but recommended for small labs
Kernel modules & sysctl	✅	Needed for networking & bridging

✅ Step 1 — Update and install dependencies
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl apt-transport-https vim git

1️⃣ — Time synchronization (important for TLS & raft)
sudo apt install -y chrony
sudo systemctl enable --now chrony

✅ Step 1 — Check Which NTP Service Is Running
Run:
systemctl status chrony

If not installed, check:
systemctl status systemd-timesyncd

Only one should be active.

✅ Step 2 — Force Immediate Time Sync
If using chrony:
sudo chronyc makestep

Then check:
timedatectl

You should now see:
System clock synchronized: yes

If using systemd-timesyncd:
sudo systemctl restart systemd-timesyncd

Then wait ~10 seconds and check again:
timedatectl

✅ Step 3 — Verify NTP Sources (Chrony)
If using chrony:
chronyc sources -v

You should see reachable servers marked with ^* or ^+.

✅ Confirm Real Sync Status
Run:
chronyc tracking

You should see something like:
Leap status : Normal
System time : ...

If Leap status: Normal → you are synchronized.

2️⃣ Disable Swap
#Check swap on docker swarm node
swapon –show
#Disable swap (Docker requirement) -> !!! On Docker Swarm Nodes
sudo swapoff -a
sudo sed -i '/swap/d' /etc/fstab
swapon –show

```
sudo swapoff -a
swapon --show
```

Verify:

```
free -h

              total        used        free      shared  buff/cache   available
Mem:          3.8Gi       1.2Gi     189Mi       32Mi       2.7Gi       2.6Gi
Swap:            0B          0B          0B
```

3️⃣ Kernel Modules & Sysctl (Networking Requirements)

```
#Run on all nodes:
sudo modprobe br_netfilter
sudo tee /etc/modules-load.d/k8s.conf <<EOF
br_netfilter
EOF

sudo tee /etc/sysctl.d/k8s.conf <<EOF
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
EOF

sudo sysctl --system
```

✅ This ensures pods can communicate across nodes and networking works correctly.

4️⃣ Install Container Runtime

Kubernetes requires a container runtime. The most common are:
containerd (lightweight, modern, recommended)
Docker (legacy, still supported, but containerd is preferred)

For a small lab, containerd is simple and works fine.

```
#Install containerd (on all nodes):
sudo apt update
sudo apt install -y containerd

#Configure systemd cgroup driver (matches kubelet):
sudo mkdir -p /etc/containerd
sudo containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd
sudo systemctl enable containerd
```

Verify:

```
containerd --version
```

Now Kubernetes will use containerd as the runtime.

5️⃣ Kubernetes Packages (kubeadm, kubelet, kubectl)

```
#Install kubeadm, kubelet, and kubectl on All Nodes
# Remove Old Repo (if exists)
sudo rm -f /etc/apt/sources.list.d/kubernetes.list
```

# Add New Kubernetes Repository (Official Method)
sudo apt update
sudo apt install -y apt-transport-https ca-certificates curl gpg

Add the GPG key:
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | \
sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

Add the repo:
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' | \
sudo tee /etc/apt/sources.list.d/kubernetes.list

# Remove hold
sudo apt-mark unhold kubelet kubeadm kubectl

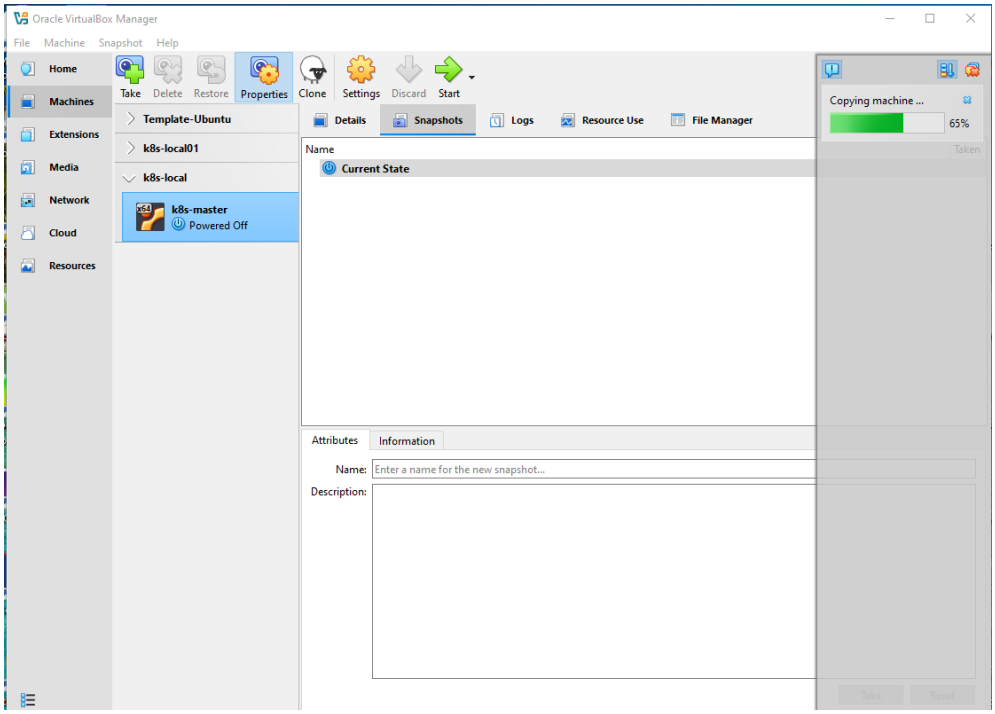
# Install again
sudo apt install -y kubelet kubeadm kubectl

# Hold them again (Kubernetes best practice)
sudo apt-mark hold kubelet kubeadm kubectl

🔎 Verify Installation:
kubeadm version
kubectl version –client
kubelet --version

Do NOT run “kubeadm init …” until all prerequisites (swap off, container runtime, networking, hostnames) are complete.
sudo poweroff

Clone VM k8s-master to k8s-worker1



Power on cloned VM and change hostname and IP:
Hostname=k8s-worker1

IP=192.168.59.11

On VM set:
sudo nano /etc/hostname
sudo nano /etc/hosts

# Set static IP
sudo nano /etc/netplan/01-network-manager-all.yaml

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: false
      addresses:
        - 192.168.59.11/24
```

#Apply Changes
sudo netplan generate

If getting ERROR:

#Apply Changes
sudo netplan generate
** (generate:1513): WARNING **: 16:07:00.927: Permissions for /etc/netplan/01-network-manager-all.yaml are too open. Netplan configuration should NOT be accessible by others.

sudo chmod 600 /etc/netplan/01-network-manager-all.yaml
sudo netplan generate
sudo netplan apply
ip a

sudo nano /etc/netplan/01-network-manager-all.yaml
Change it to:

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: false
      addresses:
        - 192.168.59.11/24
```

Save and exit.

# Apply Changes Properly
sudo netplan generate
sudo netplan apply

If you are connected over SSH, use this instead:
sudo netplan try

It will auto-rollback if networking breaks.

🔍 **Why This Happened**

- Ubuntu **Desktop** → uses NetworkManager
- Ubuntu **Server** → uses systemd-networkd
- Cloning the VM does not change that

You tried to use Desktop-style configuration on Server.

🔎 **Verify It Works**

Check interfaces:
ip a

Check routing:
ip route

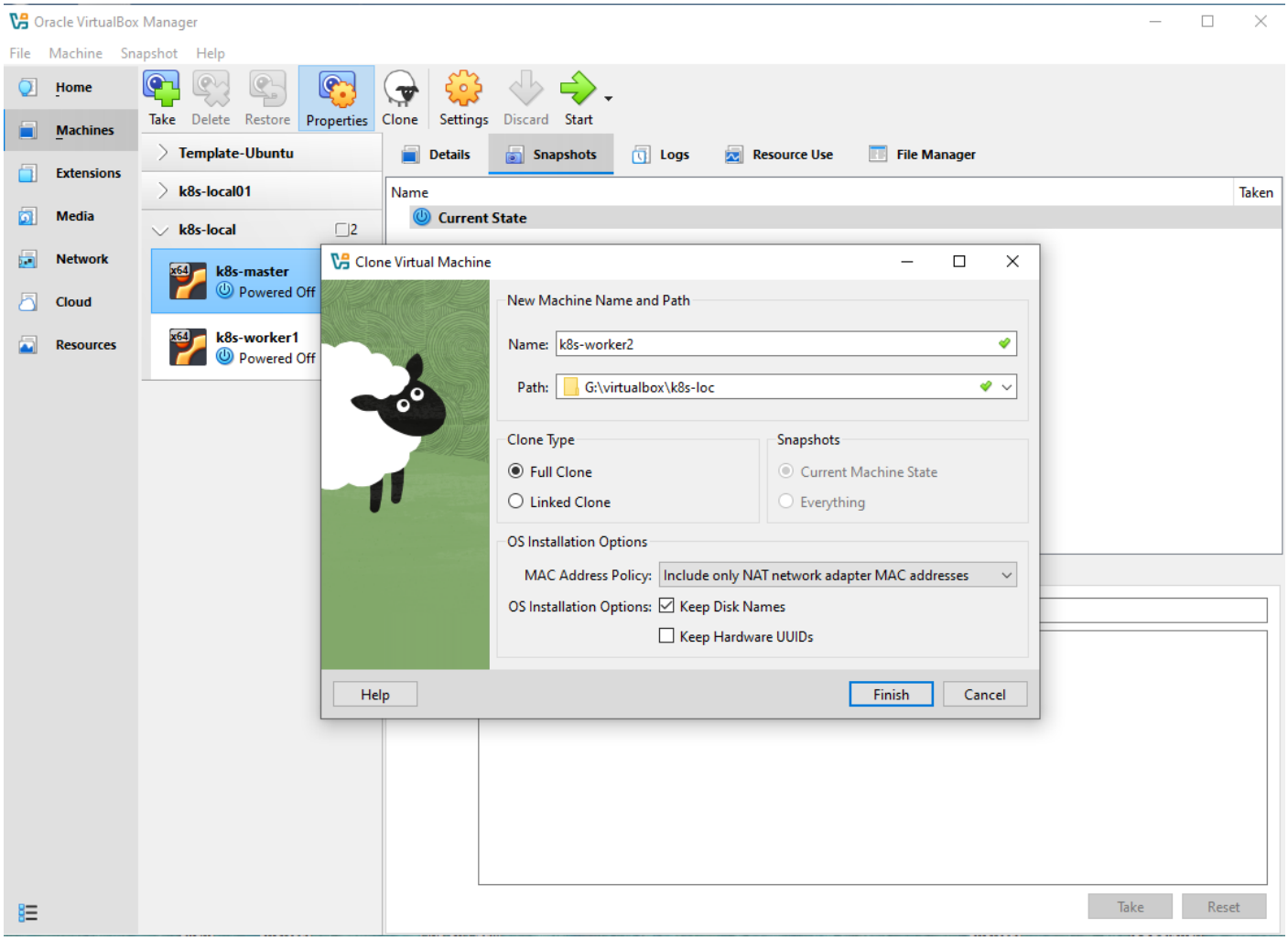
Check service:
systemctl status systemd-networkd

🎯 **Important (Cloned VM Warning)**

After cloning Ubuntu VMs, sometimes you must regenerate machine ID:
sudo rm /etc/machine-id
sudo systemd-machine-id-setup

This prevents networking conflicts between clones.

Clone master to worker2



Change:
sudo nano /etc/hostname
sudo nano /etc/hosts

Set static IP:
sudo nano /etc/netplan/01-network-manager-all.yaml

```
network:
 version: 2
 renderer: NetworkManager
 ethernets:
  enp0s3:
   dhcp4: true
  enp0s8:
   dhcp4: false
   addresses:
    - 192.168.59.12/24
```

# Apply Changes
sudo netplan generate

** (generate:1513): WARNING **: 16:07:00.927: Permissions for /etc/netplan/01-network-manager-all.yaml are too open. Netplan configuration should NOT be accessible by others.

```
sudo chmod 600 /etc/netplan/01-network-manager-all.yaml
sudo netplan generate
sudo netplan apply
ip a
```

If have duplicate IP:

```
$ ls /etc/netplan/
01-network-manager-all.yaml  50-cloud-init.yaml
sudo mv /etc/netplan/50-cloud-init.yaml /etc/netplan/50-cloud-init.yaml.old
$ nano cat /etc/netplan/01-network-manager-all.yaml
$ sudo nano /etc/netplan/01-network-manager-all.yaml
```

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: false
      addresses:
        - 192.168.59.12/24
```

```
$ sudo cat /etc/netplan/50-cloud-init.yaml
```

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: false
      addresses:
        - 192.168.59.10/24
```

```
$ sudo mv /etc/netplan/50-cloud-init.yaml  /etc/netplan/50-cloud-init.yaml.old
$ sudo netplan generate
$ sudo netplan apply
$ ip a
```

🎯 **Important (Cloned VM Warning)**

After cloning Ubuntu VMs, sometimes you must regenerate machine ID:

```
sudo rm /etc/machine-id
sudo systemd-machine-id-setup
```

This prevents networking conflicts between clones.

To test connections between VMs, open ssh connection to each and try to ping all VMs by name from ssh connection:

```
ping k8s-master
ping k8s-worker1
ping k8s-worker2
```

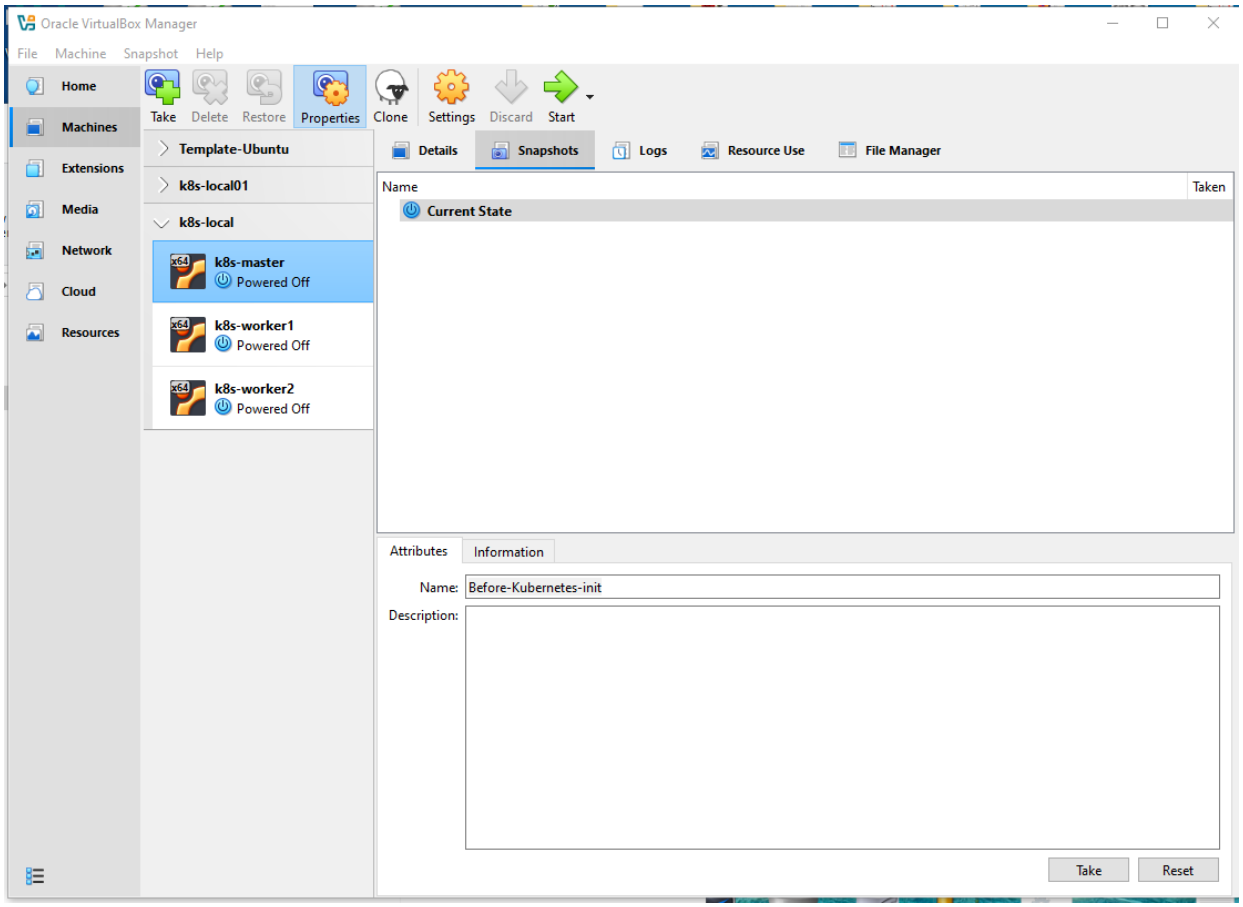
Check ip by:

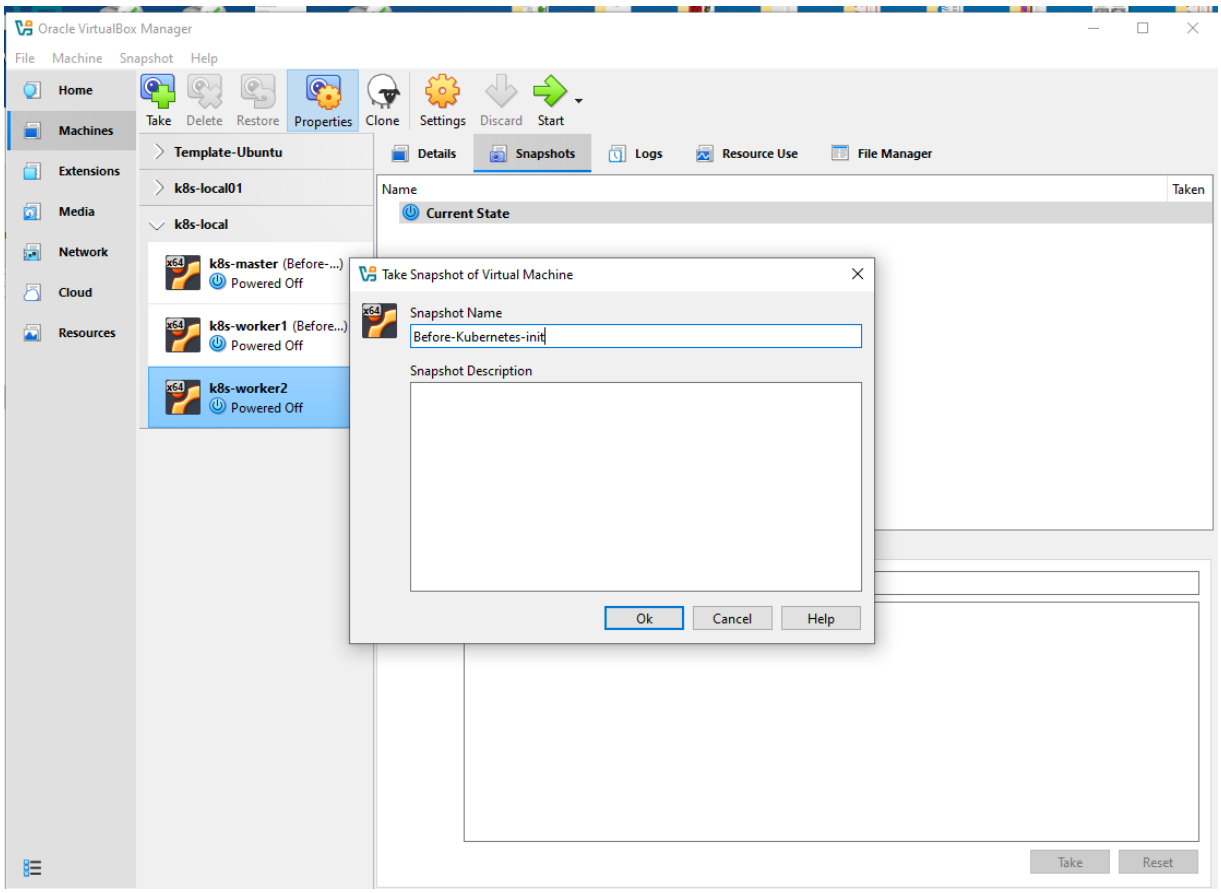
```
ip a
```

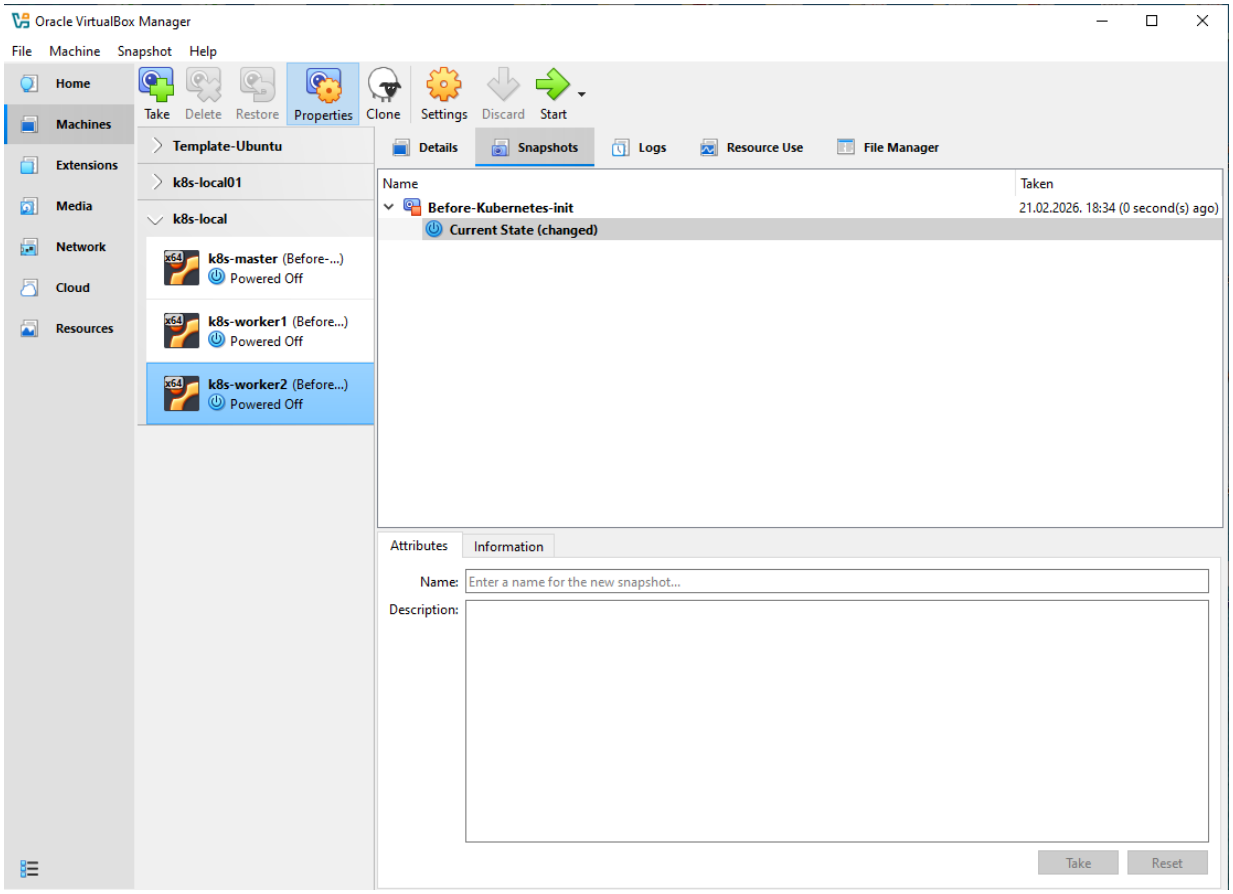
Prepare to start Kubernetes configuration

#Initialize Kubernetes Master
On each node:
sudo sysctl -w net.ipv4.ip_forward=1
sudo kubeadm config images pull
sudo poweroff

Create snapshot for each VM
Before-Kubernetes-init







Start all VMs.
ssh to each VM

Start Kubernetes configuration

On k8s-master:

✅ **Flannel – master and worker nodes minimum 4 GB RAM**

✅ **kubeadm init for Flannel**

```
sudo kubeadm init \
 --apiserver-advertise-address=192.168.59.10 \
 --pod-network-cidr=10.244.0.0/16
```

After init:
kubectl apply -f https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.ym

✅ **Calico – master and worker nodes min 4 GB RAM**

✅ **kubeadm init for Calico**

```
sudo kubeadm init \
 --apiserver-advertise-address=192.168.59.10 \
 --pod-network-cidr=192.168.0.0/16
```

After init:
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.27.2/manifests/calico.yaml

✅ **Cilium – master and worker nodes min 8 GB RAM**

✅ **kubeadm init for Cilium**

```
sudo kubeadm init \
 --apiserver-advertise-address=192.168.59.10 \
 --pod-network-cidr=10.0.0.0/16
```

Then install Cilium via CLI:

```
curl -LO https://github.com/cilium/cilium-cli/releases/latest/download/cilium-linux-amd64.tar.gz
sudo tar xzvf cilium-linux-amd64.tar.gz -C /usr/local/bin
cilium install
```

✅ Run on worker nodes:

✅ Flannel:

```
sudo kubeadm join 192.168.59.10:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>
```

Run on master:
Run this as your normal user (NOT root):

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

```
kubectl get nodes
NAME        STATUS    ROLES        AGE    VERSION
k8s-master   NotReady  control-plane  5m12s  v1.29.15
k8s-worker1  NotReady  <none>       3m22s  v1.29.15
k8s-worker2  NotReady  <none>       5s     v1.29.15
```

After init:
kubectl apply -f https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.ym

Verify:
kubectl get pods -A
or
kubectl get pods -A -w

Wait until:
calico-node
calico-kube-controllers
are Running.

kubectl get nodes

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	**Ready**	control-plane	9m51s	v1.29.15
k8s-worker1	**Ready**	<none>	7m51s	v1.29.15
k8s-worker2	**Ready**	<none>	7m30s	v1.29.15

Test simple deployment
kubectl create deployment nginx --image=nginx
kubectl get pods -o wide
kubectl get svc

run Full test
🧪 **1. BusyBox / Alpine (Basic Network Test)**
Great for basic **ping, DNS, service, and cross-pod reachability tests**.
Deploy:
kubectl run busybox --image=busybox --restart=Never -- sleep 3600

Then exec in:
kubectl exec -it busybox -- sh

Inside the container you can test:
ping <pod-ip>
wget —no-verbose http://<service-ip>
nslookup kubernetes.default

✓ Perfect for initial connectivity testing.

🔷 **2. nginx (Simple Web Server)**
Good for service and HTTP traffic test.

Deploy:
kubectl create deployment nginx --image=nginx
kubectl expose deployment nginx --port=80

Test it:
kubectl get svc nginx

Then from another pod:
wget --timeout=5 --tries=1 http://nginx

This ensures networking + service routing works.

🐸 **3. netshoot (Network Debugging Utility)**
An excellent **network troubleshoot tool** — includes tools like:

- curl
- ping
- traceroute
- dig
- tcpdump
- iperf

Deploy:
kubectl run netshoot --image=nicolaka/netshoot --restart=Never -- sleep 3600

Exec in:
kubectl exec -it netshoot -- bash

Then inside:
ip a
ping <IP>
curl http://nginx

This is much better for diagnosing connectivity issues.

🔁 **4. Demo Apps (Microservices)**
If you want a real app that exercises networking more:

- httpbin — simple HTTP echo server
- whoami — returns caller info
- echoserver — Google's sample

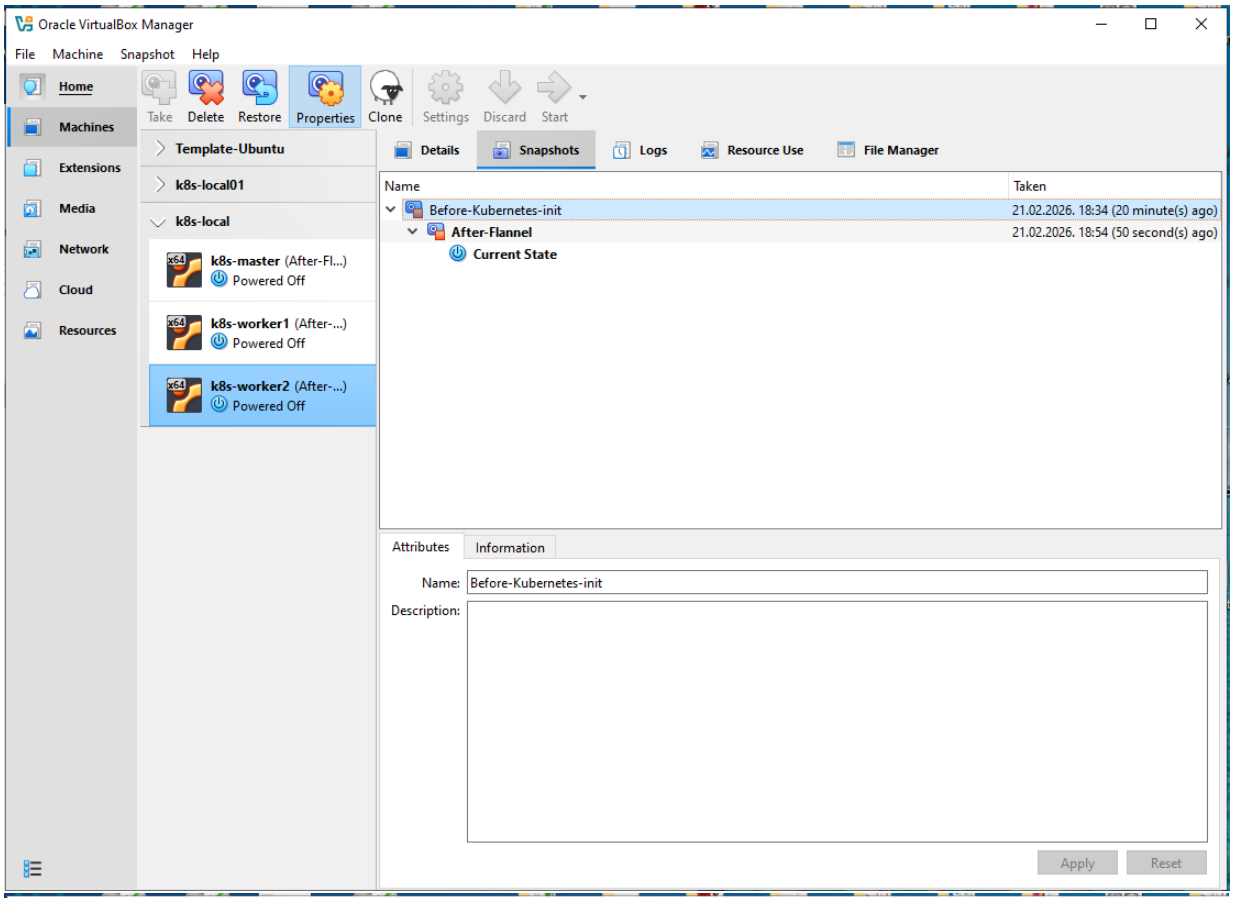
Deploy example:
kubectl run whoami --image=containous/whoami --port=80
kubectl expose deployment whoami --port=80

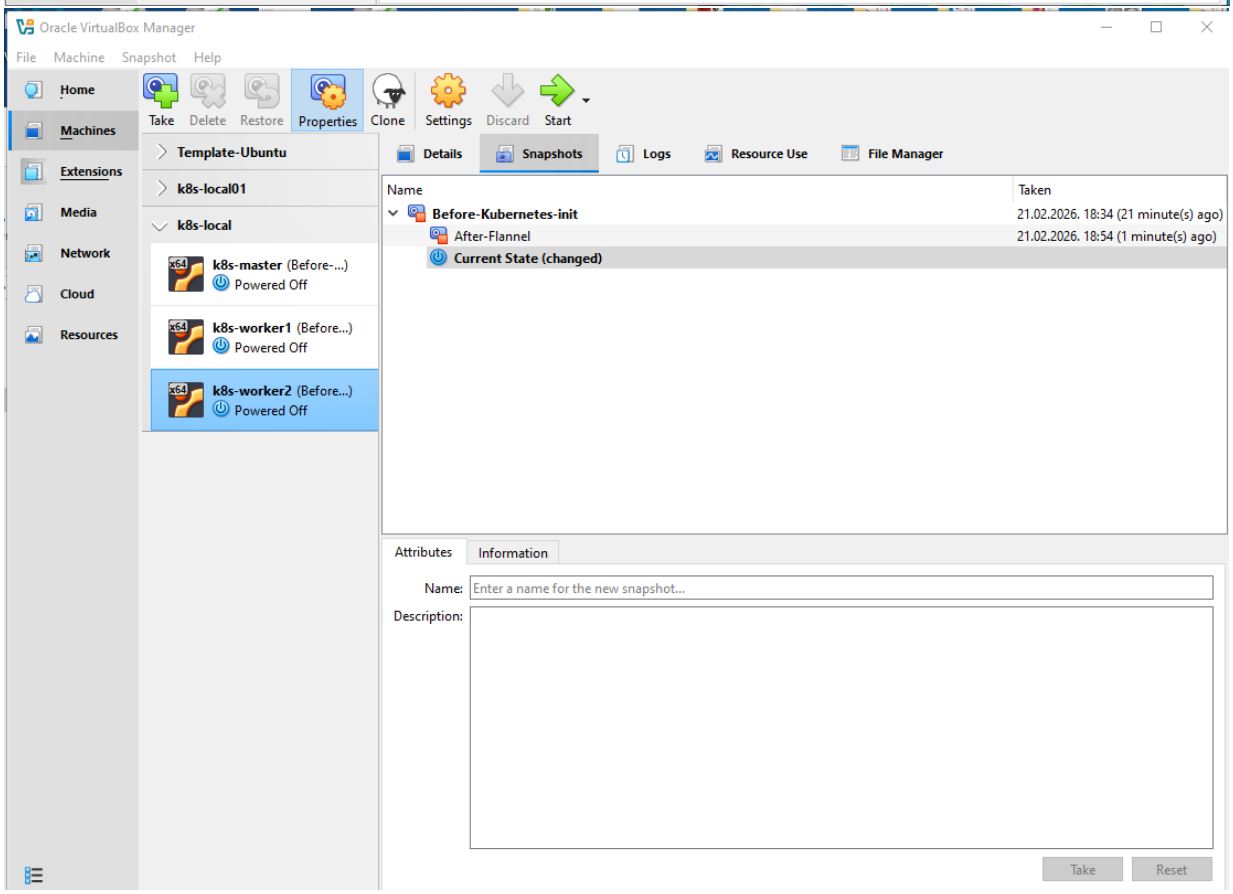
Then test:
curl http://whoami

On k8s-worker2 node execute:
sudo poweroff
On k8s-worker1 node execute:
sudo poweroff
On k8s-master node execute:
sudo poweroff

Take snapshot of the current state as "After-Flannel"

Inside virtualbox restore status of VMs to snapshot “Before-Kubernetes-init”





✅ Calico configuration:

On master node run:

```
sudo kubeadm init \
  --apiserver-advertise-address=192.168.59.10 \
  --pod-network-cidr=192.168.0.0/16
```

...

To start using your cluster, you need to run the following as a regular user:

```
  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Alternatively, if you are the root user, you can run:
export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.

Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on each as root:

```
sudo kubeadm join 192.168.59.10:6443 --token mu9y8b.uxwceohycefnpl1x \
      --discovery-token-ca-cert-hash
sha256:b2c77b5e8c46af2e6f9f75dccbc382aa82b865bdbb17e5829233cd967c27aae3
```

After init:
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.27.2/manifests/calico.yaml

Run on master:

Run this as your normal user (NOT root):

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

```
kubectl get nodes
NAME        STATUS    ROLES        AGE    VERSION
k8s-master   NotReady  control-plane  5m12s  v1.29.15
k8s-worker1  NotReady  <none>       3m22s  v1.29.15
k8s-worker2  NotReady  <none>       5s     v1.29.15
```

After init:
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.27.2/manifests/calico.yaml

Verify:
kubectl get pods -A

Wait until:
calico-node
calico-kube-controllers

are Running.

#Test simple deployment
kubectl create deployment nginx --image=nginx
kubectl get pods -o wide
kubectl get svc

On k8s-worker2 node execute:
sudo poweroff
On k8s-worker1 node execute:
sudo poweroff
On k8s-master node execute:
sudo poweroff

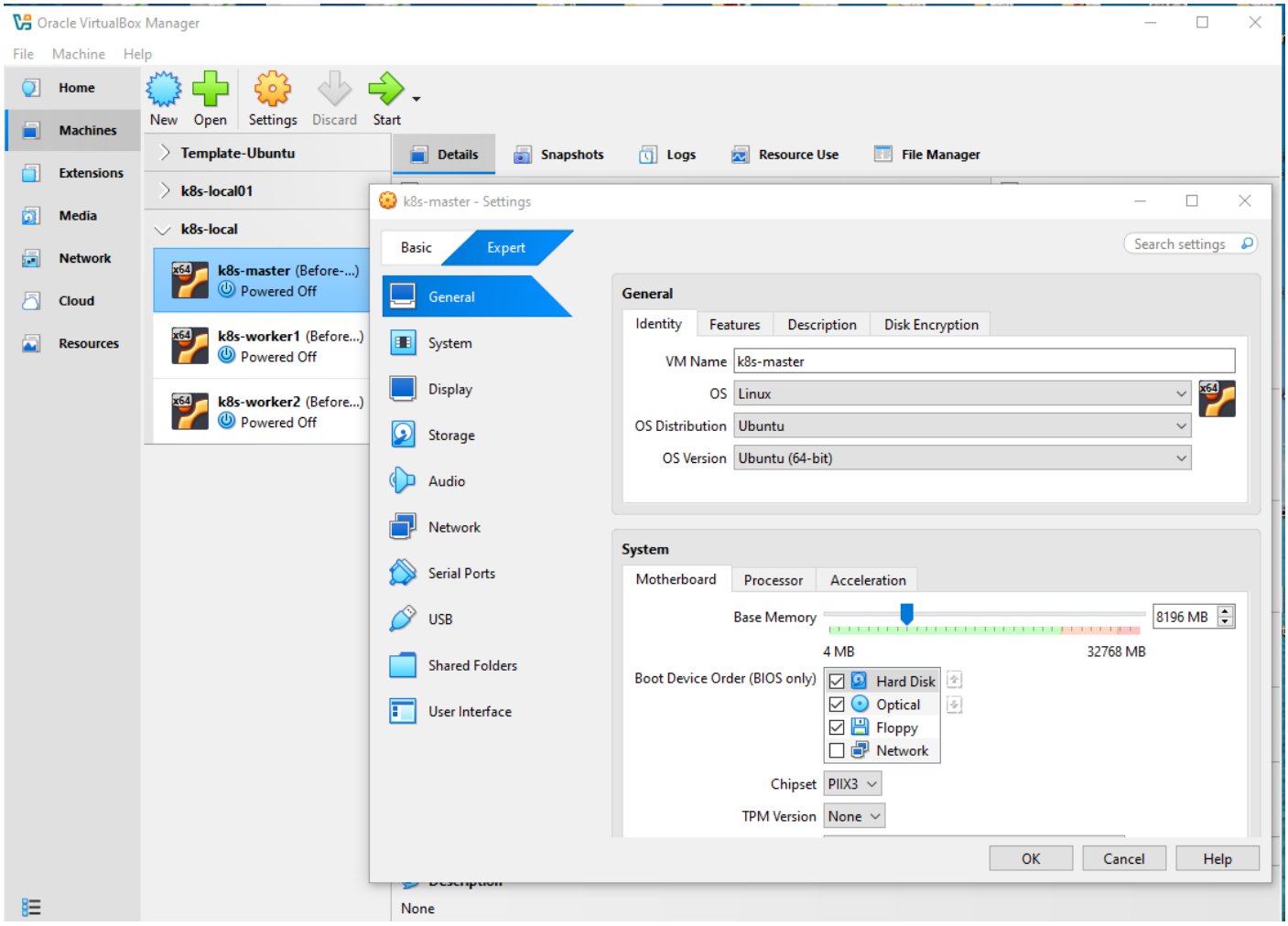
Take snapshot of the current state as “After-Calico “

Inside virtualbox restore status of VMs to snapshot “Before-Kubernetes-init”

✅ Cilium

✅ Cilium – master and worker nodes min 8 GB RAM

On each VM set memory to 8192 MB



```
sudo kubeadm init \
  --apiserver-advertise-address=192.168.59.10 \
  --pod-network-cidr=10.0.0.0/16
```

To start using your cluster, you need to run the following as a regular user:

```
  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Alternatively, if you are the root user, you can run:
export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on each as root:
On workers:

```
sudo kubeadm join 192.168.59.10:6443 --token negb79.sztj0sgy9ffsxe2z \
      --discovery-token-ca-cert-hash
sha256:b25b4bc583a6d72f5436c1a84df89e2497ab06bd647ac12b1a969946d64f3d6c
```

kubectl get nodes

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	**NotReady**	control-plane	3m13s	v1.29.15
k8s-worker1	**NotReady**	<none>	2m11s	v1.29.15
k8s-worker2	**NotReady**	<none>	115s	v1.29.15

On master:
Then install Cilium via CLI:

```
curl -LO https://github.com/cilium/cilium-cli/releases/latest/download/cilium-linux-amd64.tar.gz
sudo tar xzvf cilium-linux-amd64.tar.gz -C /usr/local/bin
cilium install
```

Or, using kubectl manifest:
kubectl create -f https://raw.githubusercontent.com/cilium/cilium/v1.14/install/kubernetes/quick-install.yaml

```
cilium install \
  --set kubeProxyReplacement=disabled
```

If you have an issue to start Cilium, follow the steps:

```
cilium uninstall
kubectl delete ds cilium -n kube-system --ignore-not-found
kubectl delete deployment cilium-operator -n kube-system --ignore-not-found
kubectl delete cm cilium-config -n kube-system --ignore-not-found
kubectl delete sa cilium -n kube-system --ignore-not-found
kubectl delete crd ciliumnetworkpolicies.cilium.io --ignore-not-found
kubectl delete crd ciliumnodes.cilium.io --ignore-not-found
kubectl get pods -n kube-system | grep cilium
```

On all nodes run:

```
sudo kubeadm reset -f
```

On the master:

```
sudo systemctl restart containerd
sudo systemctl restart kubelet
sudo rm -rf /etc/cni/net.d/*
sudo kubeadm init   --apiserver-advertise-address=192.168.59.10   --pod-network-cidr=192.168.0.0/16

mkdir -p $HOME/.kube
sudo cp -f /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

kubectl config view
kubectl get nodes

curl -LO https://github.com/cilium/cilium-cli/releases/latest/download/cilium-linux-amd64.tar.gz
sudo tar xzvf cilium-linux-amd64.tar.gz -C /usr/local/bin
cilium install
```

Verify:

```
kubectl get pods -A
or
kubectl get pods -A -w
```

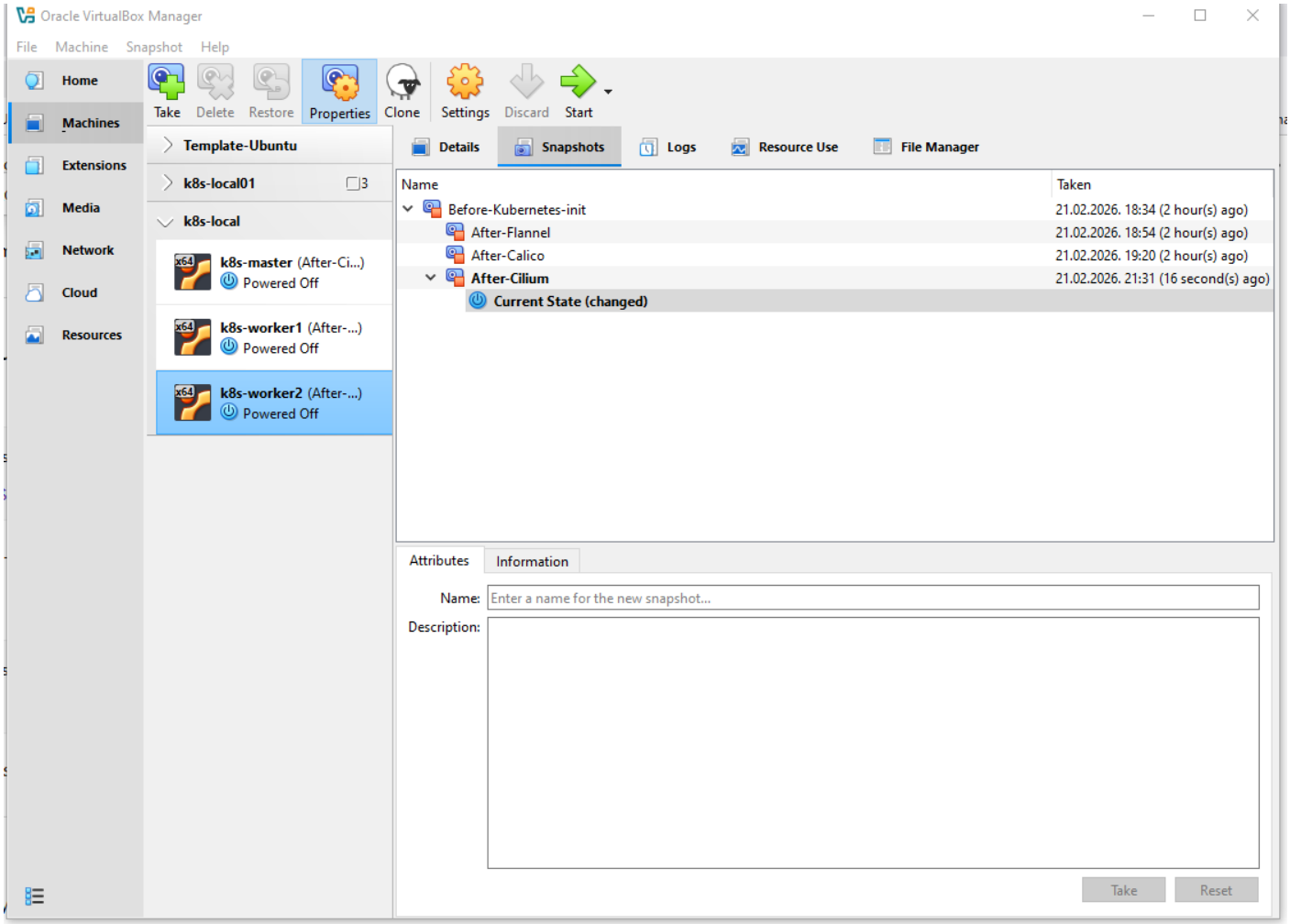
All cilium-* pods → Running
Nodes → Ready
coredns-* → Running

```
kubectl get pods -n kube-system


kubectl get nodes
NAME          STATUS   ROLES           AGE    VERSION
k8s-master    Ready    control-plane   10m    v1.29.15
k8s-worker1   Ready    <none>          7m54s  v1.29.15
k8s-worker2   Ready    <none>          7m27s  v1.29.15

$ cilium status
    /¯¯\
 /¯¯\__/¯¯\    Cilium:         OK
 \__/¯¯\__/    Operator:       OK
 /¯¯\__/¯¯\    Envoy DaemonSet:   OK
 \__/¯¯\__/    Hubble Relay:     disabled
    \__/       ClusterMesh:      disabled

DaemonSet          cilium               Desired: 3, Ready: 3/3, Available: 3/3
DaemonSet          cilium-envoy         Desired: 3, Ready: 3/3, Available: 3/3
Deployment         cilium-operator      Desired: 1, Ready: 1/1, Available: 1/1
Containers:        cilium               Running: 3
                   cilium-envoy         Running: 3
                   cilium-operator      Running: 1
                   clustermesh-apiserver
                   hubble-relay
Cluster Pods:      2/2 managed by Cilium
Helm chart version:   1.19.0
Image versions     cilium
quay.io/cilium/cilium:v1.19.0@sha256:be9f8571c2e114b3e12e41f785f2356ade703b2eac936aa878805565f0468c60: 3
                   cilium-envoy     quay.io/cilium/cilium-envoy:v1.35.9-1768828720-
c6e4827ebca9c47af2a3a6540c563c30947bae29@sha256:696582a3391ce05a62edb4140e6a99f774351f363f5b5d7f1581f3a24443024
9: 3
                   cilium-operator   quay.io/cilium/operator-
generic:v1.19.0@sha256:5b04006015e5800307dc6314676edc4c0bb7ac2fc7848be2b94b43bb030ab648: 1
```

#Test simple deployment
kubectl create deployment nginx --image=nginx
kubectl get pods -o wide
kubectl get svc

On k8s-worker2 node execute:
sudo poweroff
On k8s-worker1 node execute:
sudo poweroff
On k8s-master node execute:
sudo poweroff

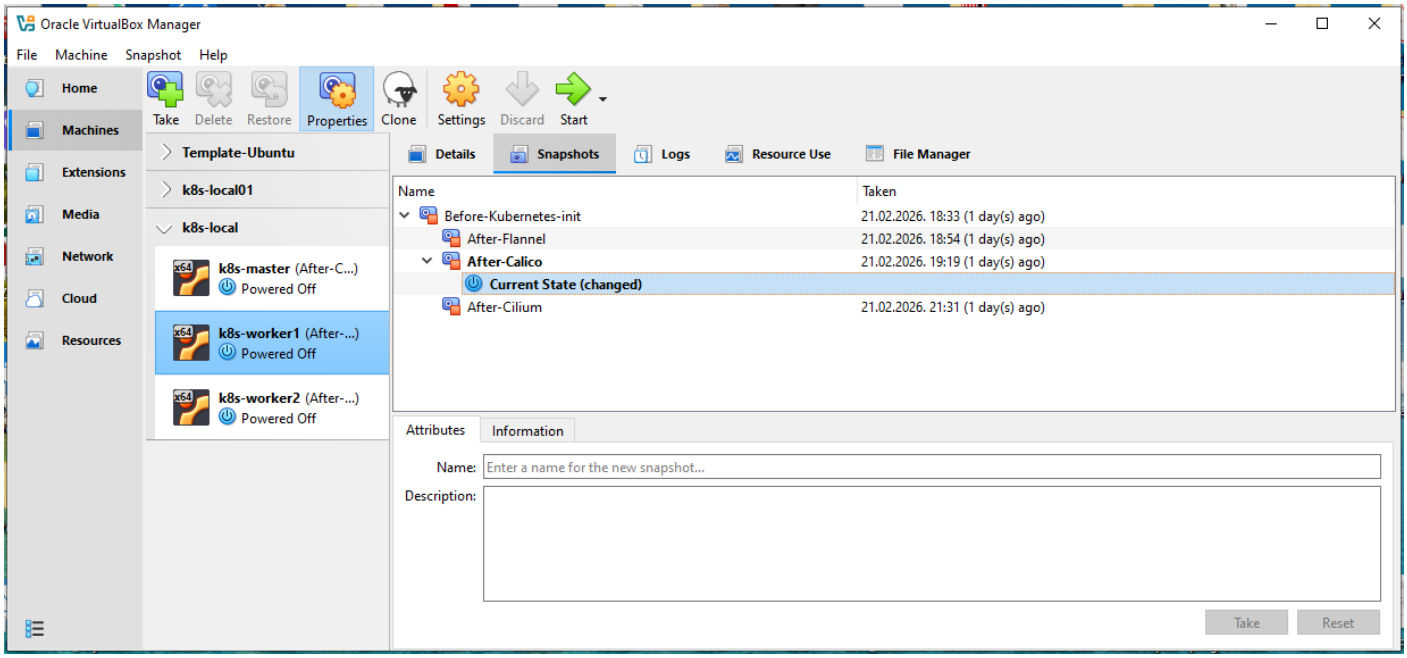
Take snapshot of the current state as “After-Cilium”



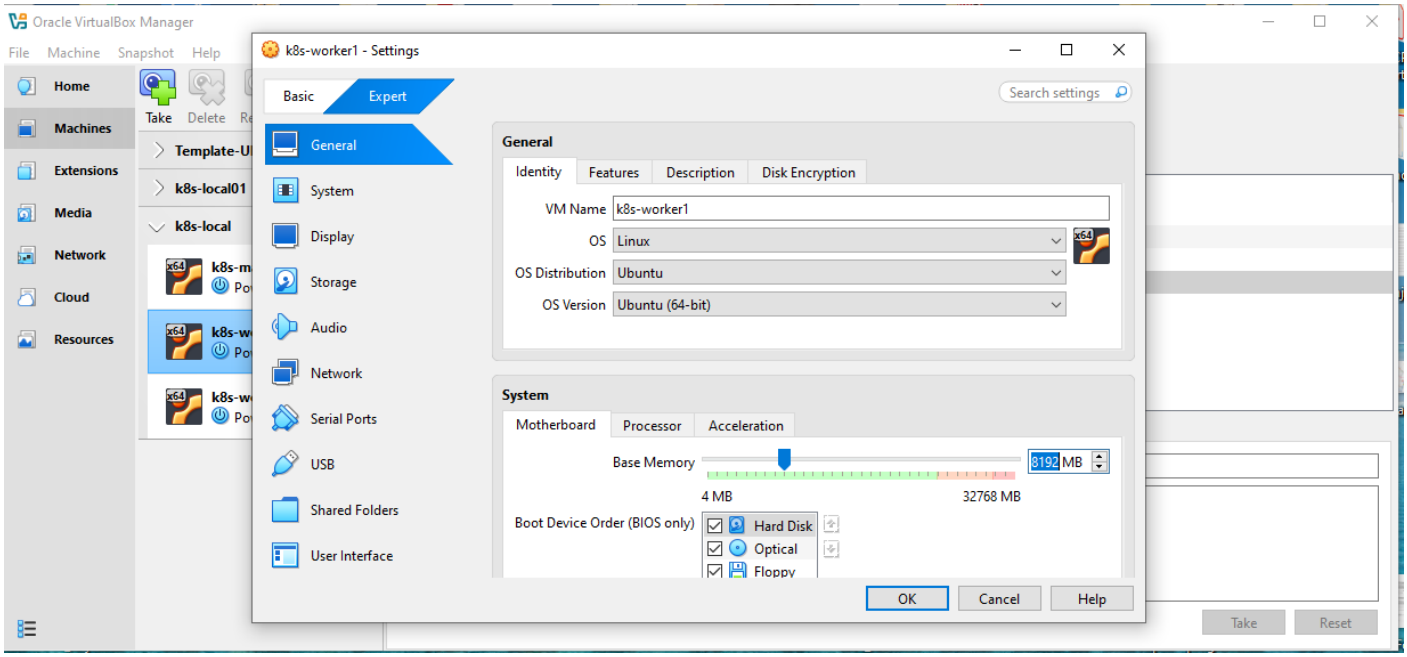
✅ Calico + Istio installation

Note: Istio isn’t Container Network Interface (CNI) and as the starting point to install Istio we will use installed Calico configuration.

#Use Calico snapshot to continue installation
Restore snapshot “After-Calico”



Change memory for all VMs for Kubernetes cluster to 8192 MB



Power On VMs.

◆ Important: Calico + Istio Compatiblity
Calico works very well with Istio.
You just need to make sure:
Calico is running in kube-proxy replacement mode OR normal mode (both fine)
You do NOT enable Istio CNI unless you specifically want it
You disable Istio's internal CNI chaining unless needed
In most cases → standard Istio install works without modification.

◆ Download Istio and Install Istio CLI
On your control-plane VM:
curl -L https://istio.io/downloadIstio | sh -
cd istio-*

#export PATH=$PWD/bin:$PATH
sudo mv bin/istioctl /usr/local/bin/

Verify:
istioctl version

🔷 Resource Consideration (Important for 3 VM Lab)
Istio adds overhead:
Each pod gets:
Envoy sidecar (~100–200MB RAM)
Extra CPU usage
Make sure your worker nodes have:
Minimum recommended per worker:
4 vCPU
8GB RAM

If your VMs are small, use:
istioctl install --set profile=minimal -y

🔷 Install Istio (demo profile)
istioctl install --set profile=demo -y

OR for production-style minimal install:
istioctl install --set profile=default -y

🔷 Optional (Advanced): Use Istio CNI Plugin
If you want to avoid init containers modifying iptables:
Install Istio with CNI:
istioctl install --set profile=default --set components.cni.enabled=true -y

But this is optional.

With Calico, you usually don't need it.
What this installs:
istiod (control plane)
ingress gateway
sidecar injector

Verify:
kubectl get pods -n istio-system

Flow becomes:
Pod → Envoy Sidecar → Calico CNI → Network
Calico handles:
Pod networking
Network policy
eBPF dataplane

Istio handles:
mTLS
Traffic routing
Observability
Service mesh

They operate at different layers, so no conflict.

🔷 Enable Automatic Sidecar Injection
kubectl label namespace default istio-injection=enabled

Or create a dedicated namespace:
kubectl create namespace myapp
kubectl label namespace myapp istio-injection=enabled

Deploy a Test Application
kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml

Verify pods:
kubectl get pods
or
kubectl get pods -w

You should see 2 containers per pod:
App container
istio-proxy (Envoy sidecar)

🔷 Expose via Istio Ingress Gateway
kubectl apply -f samples/bookinfo/networking/bookinfo-gateway.yaml

🔷 Install Ingress Gateway Only (Recommended)
You don't need to reinstall everything.
Run:

```
istioctl install \
  --set profile=default \
  --set components.ingressGateways[0].name=istio-ingressgateway \
  --set components.ingressGateways[0].enabled=true \
  -y
```

Get ingress IP:
kubectl get svc istio-ingressgateway -n istio-system

🔷 Special Note for Calico Users
If Calico is running in:
kube-proxy replacement strict mode

Ensure:
helm get values Calico -n kube-system

Look for:
kubeProxyReplacement: strict

Istio works fine with this.

🔷 Verify Everything
Check Istio pods:
kubectl get pods -n istio-system

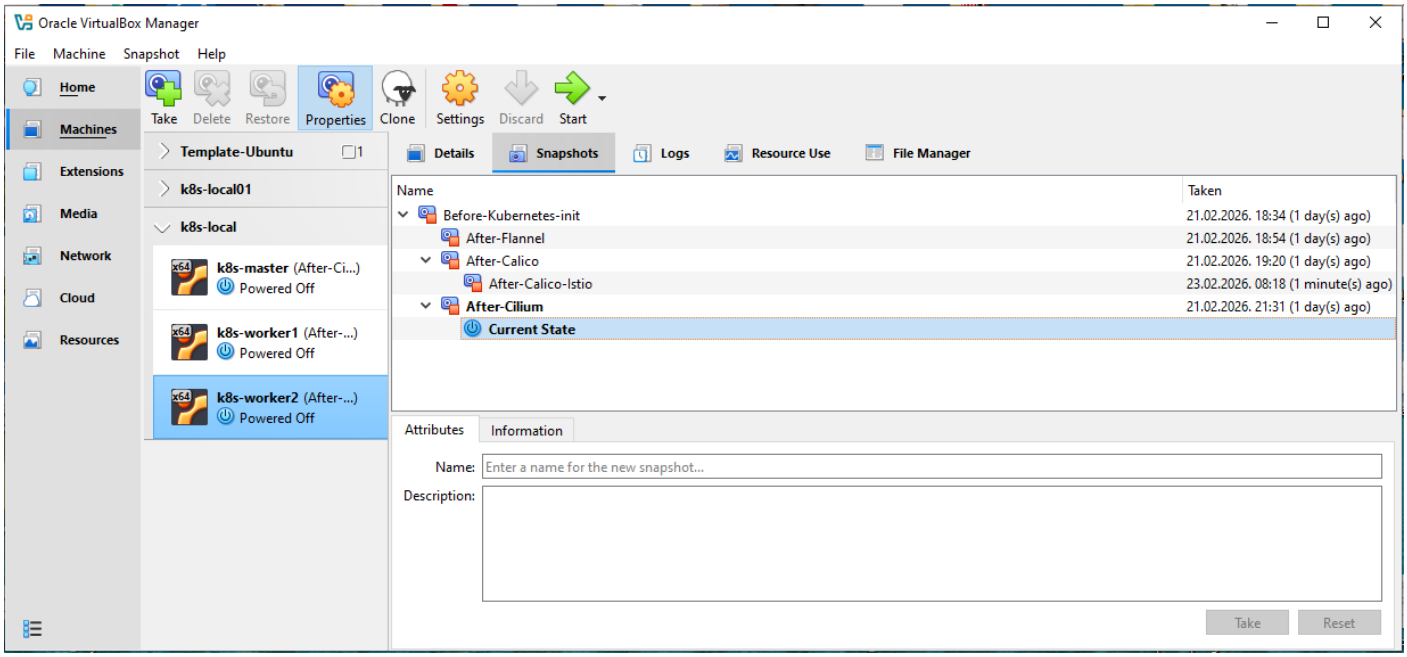
Check proxy status:
istioctl proxy-status

On each VM execute:
sudo poweroff

✅ Cilium + Istio installation

Note: Istio isn't Container Network Interface (CNI) and as the starting point to install Istio we will use installed Cilium configuration.

Restore snapshot "After- Cilium"



Power On VMs.

🔷 Cilium + Istio installation
#Use Cilium snapshot to continue installation

🔷 Important: Cilium + Istio Compatibility
Cilium works very well with Istio.
You just need to make sure:
Cilium is running in kube-proxy replacement mode OR normal mode (both fine)
You do NOT enable Istio CNI unless you specifically want it
You disable Istio's internal CNI chaining unless needed
In most cases → standard Istio install works without modification.
On master:
kubectl get nodes

🔷 Download Istio and Install Istio CLI
#On your control-plane VM:
curl -L https://istio.io/downloadIstio | sh -
cd istio-*
#export PATH=$PWD/bin:$PATH
sudo mv bin/istioctl /usr/local/bin/

Verify:
istioctl version

🔷 Resource Consideration (Important for 3 VM Lab)
Istio adds overhead:
Each pod gets:
Envoy sidecar (~100–200MB RAM)
Extra CPU usage
Make sure your worker nodes have:
Minimum recommended per worker:
4 vCPU
8GB RAM
If your VMs are small, use:
istioctl install --set profile=minimal -y

🔷 Install Istio (demo profile)
istioctl install --set profile=demo -y
OR for production-style minimal install:
istioctl install --set profile=default -y

🔷 Optional (Advanced): Use Istio CNI Plugin
If you want to avoid init containers modifying iptables:
Install Istio with CNI:
istioctl install --set profile=default --set components.cni.enabled=true -y
But this is optional.
With Cilium, you usually don't need it.

What this installs:
istiod (control plane)
ingress gateway
sidecar injector

Verify:
kubectl get pods -n istio-system
Flow becomes:
Pod → Envoy Sidecar → Cilium CNI → Network
Cilium handles:
Pod networking
Network policy
eBPF dataplane

Istio handles:
mTLS
Traffic routing
Observability
Service mesh
They operate at different layers, so no conflict.

🔷 Enable Automatic Sidecar Injection
kubectl label namespace default istio-injection=enabled
Or create a dedicated namespace:
kubectl create namespace myapp
kubectl label namespace myapp istio-injection=enabled
Deploy a Test Application

kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml
Note:
To start all pods you need to wait 5+ minutes, depend how powerful is you HW

Verify pods:
kubectl get pods
or
kubectl get pods -w
You should see 2 containers per pod:
App container
istio-proxy (Envoy sidecar)

🔷 Install Ingress Gateway Only (Recommended)
You don't need to reinstall everything.
Run:

```
istioctl install \
  --set profile=default \
  --set components.ingressGateways[0].name=istio-ingressgateway \
  --set components.ingressGateways[0].enabled=true \
  -y
```

Get ingress IP:
kubectl get svc istio-ingressgateway -n istio-system

🔷 Expose via Istio Ingress Gateway
kubectl apply -f samples/bookinfo/networking/bookinfo-gateway.yaml

🔷 Special Note for Cilium Users
If Cilium is running in:
kube-proxy replacement strict mode
Ensure:
helm get values cilium -n kube-system
Look for:
kubeProxyReplacement: strict
Istio works fine with this.

🔷 Verify Everything
Check Istio pods:
kubectl get pods -n istio-system
Check proxy status:
istioctl proxy-status

✅ How To Access the Application

Step 1 — Get Worker Node IP Inside Kubernetes:
kubectl get nodes -o wide

You should see something like:
kubectl get nodes -o wide

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME
k8s-master	Ready	control-plane	35h	v1.29.15	192.168.59.10	<none>	Ubuntu 24.04.4 LTS	6.8.0-100-generic	containerd://1.7.28

```
k8s-worker1  Ready   <none>        35h  v1.29.15  192.168.59.11  <none>     Ubuntu 24.04.4 LTS  6.8.0-100-generic  containerd://1.7.28
k8s-worker2  Ready   <none>        35h  v1.29.15  192.168.59.12  <none>     Ubuntu 24.04.4 LTS  6.8.0-100-generic  containerd://1.7.28
```

Make sure those are the Host-Only NIC IPs (192.168.59.x).

Step 2 — Confirm NodePort

kubectl get svc -n istio-system

```
NAME                      TYPE           CLUSTER-IP       EXTERNAL-IP  PORT(S)                          AGE
istio-ingressgateway      LoadBalancer   10.111.163.203   <pending>    15021:32505/TCP,80:32220/TCP,443:31100/TCP  7m37s
istiod                    ClusterIP      10.97.78.27      <none>       15010/TCP,15012/TCP,443/TCP,15014/TCP       16m
istiod-revision-tag-default  ClusterIP   10.103.68.64     <none>       15010/TCP,15012/TCP,443/TCP,15014/TCP       16m
```

You see:

```
NAME                      TYPE           CLUSTER-IP       EXTERNAL-IP  PORT(S)                          AGE
istio-ingressgateway      LoadBalancer   10.111.163.203   <pending>    15021:32505/TCP,80:32220/TCP,443:31100/TCP  7m37s
```

80:32220/TCP

So HTTP traffic is on port:

32220

Step 3 — From Your Windows Browser

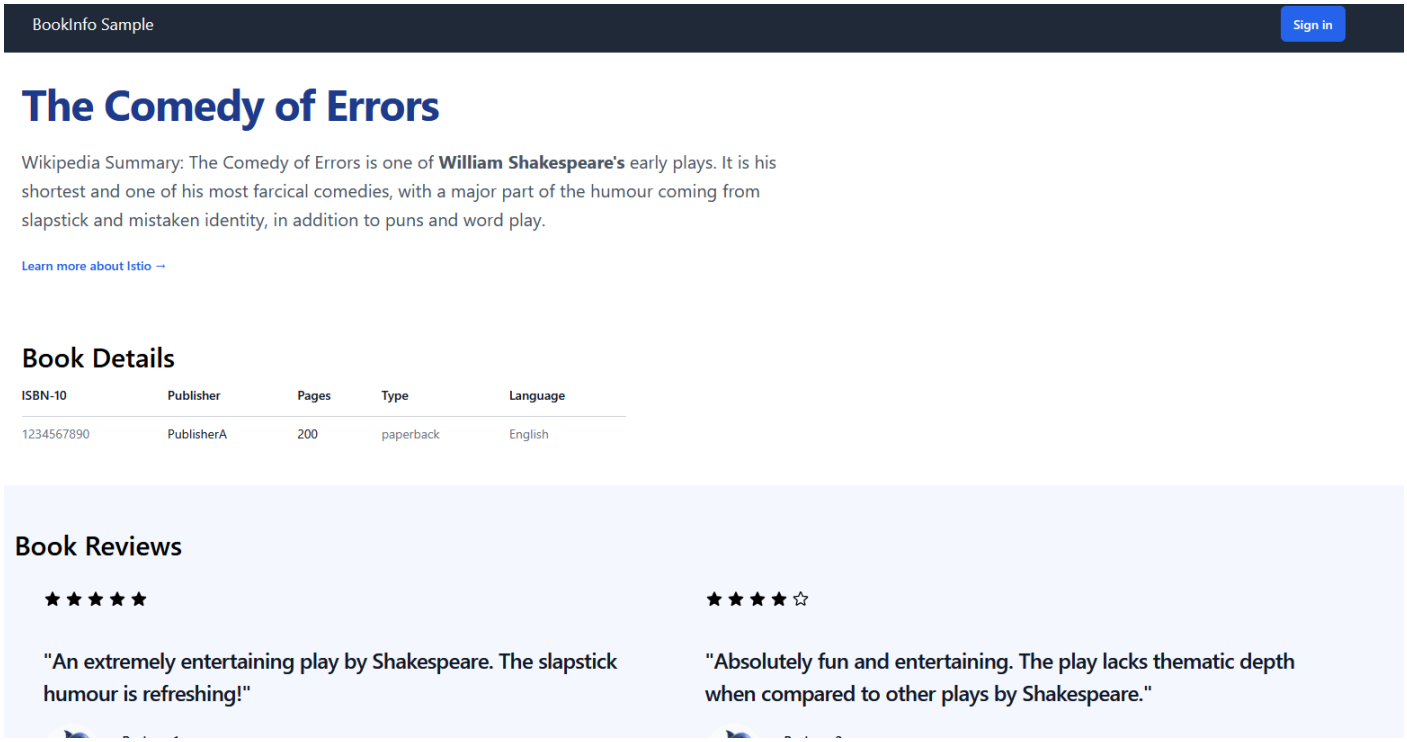
Open:

http://192.168.59.10:32220/productpage

http://192.168.59.11:32220/productpage

http://192.168.59.12:32220/productpage

That should work immediately.



Traffic path:

Windows Host

→ VirtualBox Host-Only Adapter (vboxnet0)

→ VM (192.168.56.x)

→ NodePort 31388

→ Istio Ingress Gateway

→ Bookinfo app

🚨 If It Does NOT Work
Check these 3 things:
1️⃣ Windows Firewall
Temporarily test:
Disable Windows Defender Firewall
OR
Allow inbound 31388 on Private network

2️⃣ VM Firewall (Linux inside VM)
On worker node:
sudo iptables -L
Or if using firewalld:
sudo firewall-cmd --list-all
If needed:
sudo firewall-cmd --add-port=31388/tcp --permanent
sudo firewall-cmd --reload

3️⃣ Kubernetes NodePort Binding
Check:
kubectl get svc istio-ingressgateway -n istio-system -o wide

Make sure NodePort is still 31388.

❗ IMPORTANT: Do NOT Use NAT IP
Do NOT use the NAT interface IP (like 10.0.2.x).
NAT in VirtualBox:
Allows VM → Internet
Does NOT allow Host → VM directly
Host-only network is the correct one.

🧠 Cleanest Setup Recommendation

For Kubernetes labs in VirtualBox:
Keep NAT (for internet access inside VM)
Use Host-only for cluster communication and host access
Make sure kubelet binds to 192.168.56.x interface

🎯 Final Answer
From your Windows host:
Open browser and go to:
http://<worker-node-hostonly-ip>:31388/productpage

Example:
http://192.168.56.11:31388/productpage